12c — Multi-session asset preparation and readiness driver

This notebook does not overwrite existing notebooks. It tells you which Allen sessions already have enhanced features, tensors, and embeddings, and writes a concrete to-do table for producing missing tensors/embeddings before rerunning notebooks 13–17.

Setup and asset index

```
from pathlib import Path
import os
import sys
import yaml
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# Edit this path only if your project has moved.
PROJECT_ROOT = Path(
    os.environ.get(
        "LATENT_MANIFOLD_PROJECT_ROOT",
        r"C:\Users\Peter\Documents\projects\NeuroAI\latent-manifold-v1-natural
    )
).resolve()

SRC_DIR = PROJECT_ROOT / "src"
if not PROJECT_ROOT.exists():
    raise FileNotFoundError(f"PROJECT_ROOT does not exist: {PROJECT_ROOT}")
if not SRC_DIR.exists():
    raise FileNotFoundError(f"src directory does not exist: {SRC_DIR}")
if str(SRC_DIR) not in sys.path:
    sys.path.insert(0, str(SRC_DIR))
os.chdir(PROJECT_ROOT)

cfg_path = PROJECT_ROOT / "configs" / "publication_upgrade.yaml"
if not cfg_path.exists():
    raise FileNotFoundError(f"Could not find publication config: {cfg_path}")
with cfg_path.open("r", encoding="utf-8") as f:
    pub_cfg = yaml.safe_load(f)
cfg = pub_cfg
```

```
# Versioned namespace. This prevents accidental overwriting of older publicati
PUBLICATION_RUN_LABEL = os.environ.get("PUBLICATION_RUN_LABEL", "publication_u
ALLOW_CANONICAL_PUBLICATION_WRITE = os.environ.get("ALLOW_CANONICAL_PUBLICATIO
if PUBLICATION_RUN_LABEL == "publication_upgrade" and not ALLOW_CANONICAL_PUBL
    raise RuntimeError(
        "Refusing to write into the canonical publication_upgrade namespace. "
        "Use a versioned PUBLICATION_RUN_LABEL or set ALLOW_CANONICAL_PUBLICAT
    )

class ProjectPaths:
    def __init__(self, root: Path, run_label: str):
        self.root = root
        self.configs_dir = root / "configs"
        self.data_dir = root / "data"
        self.external_dir = root / "data" / "external"
        self.raw_dir = root / "data" / "raw"
        self.interim_dir = root / "data" / "interim"
        self.processed_dir = root / "data" / "processed"
        self.versioned_processed_dir = self.processed_dir / run_label
        self.reports_dir = root / "reports"
        self.tables_dir = root / "reports" / "tables"
        self.figures_dir = root / "reports" / "figures"
        self.html_dir = root / "reports" / "html"
        self.publication_tables_dir = root / "reports" / "tables" / run_label
        self.publication_figures_dir = root / "reports" / "figures" / run_labe
        self.manuscript_dir = root / "manuscript" / "top_journal_scaffold"
        for d in [
            self.versioned_processed_dir,
            self.publication_tables_dir,
            self.publication_figures_dir,
            self.manuscript_dir,
        ]:
            d.mkdir(parents=True, exist_ok=True)

paths = ProjectPaths(PROJECT_ROOT, PUBLICATION_RUN_LABEL)

def save_table(df, path):
    path = Path(path)
    path.parent.mkdir(parents=True, exist_ok=True)
    df.to_csv(path, index=False)
    return path

def save_figure(fig, path, dpi=300):
    path = Path(path)
    path.parent.mkdir(parents=True, exist_ok=True)
    fig.savefig(path, dpi=dpi, bbox_inches="tight")
    return path

# Upgrade helper functions shipped with this patch.
from v1_manifold_publication.multisession_core import (
```

```
    build_multisession_asset_index,
    candidate_feature_files,
    choose_best_feature_table,
    ready_sessions,
    movie_feature_targets,
    empirical_null_summary,
    safe_read_csv,
    safe_table_index,
    claim_gate_from_assets,
    ensure_table,
)

metadata_path = paths.external_dir / "allen_v1_natural_movie_experiments.csv"
asset_index = build_multisession_asset_index(paths, metadata_path=metadata_pat
save_table(asset_index, paths.publication_tables_dir / "00_multisession_asset_

print("Using PROJECT_ROOT:", PROJECT_ROOT)
print("Publication run label:", PUBLICATION_RUN_LABEL)
print("Publication tables:", paths.publication_tables_dir)
print("Publication figures:", paths.publication_figures_dir)
print("Versioned processed dir:", paths.versioned_processed_dir)
display(asset_index)
```

```
Using PROJECT_ROOT: C:\Users\Peter\Documents\projects\NeuroAI\latent-manifold
Publication run label: publication_upgrade_v3_multisession
Publication tables: C:\Users\Peter\Documents\projects\NeuroAI\latent-manifold
Publication figures: C:\Users\Peter\Documents\projects\NeuroAI\latent-manifol
Versioned processed dir: C:\Users\Peter\Documents\projects\NeuroAI\latent-man
```

	session_id	feature_file	feature_shape	
0	500855614	C:\Users\Peter\Documents\projects\NeuroAI\late...	(900, 105)	\Users\
1	500964514	C:\Users\Peter\Documents\projects\NeuroAI\late...	(900, 105)	
2	501271265	C:\Users\Peter\Documents\projects\NeuroAI\late...	(900, 105)	

Multi-session readiness report

```
# Recommended top-journal threshold: at least three fully processed neural s
MIN_TOP_JOURNAL_SESSIONS = int(os.environ.get("MIN_TOP_JOURNAL_SESSIONS", "3

asset_index = build_multisession_asset_index(paths, metadata_path=metadata_p
```

```
asset_index = build_multisession_asset_index(paths, metadata_path=metadata_p
save_table(asset_index, paths.publication_tables_dir / "12c_multisession_ass

counts = pd.DataFrame([
    {"asset": "sessions_with_features", "count": int(asset_index["has_featur
    {"asset": "sessions_with_tensors", "count": int(asset_index["has_tensor"
    {"asset": "sessions_with_embeddings", "count": int(asset_index["has_embe
    {"asset": "sessions_ready_for_full_neural_analysis", "count": int(asset_
    {"asset": "minimum_target_for_top_journal_rerun", "count": MIN_TOP_JOURN
])
save_table(counts, paths.publication_tables_dir / "12c_multisession_readines

display(asset_index)
display(counts)
```

	session_id	feature_file	feature_shape	
0	500855614	C:\Users\Peter\Documents\projects\NeuroAI\late...	(900, 105)	\Users\
1	500964514	C:\Users\Peter\Documents\projects\NeuroAI\late...	(900, 105)	
2	501271265	C:\Users\Peter\Documents\projects\NeuroAI\late...	(900, 105)	

	asset	count
0	sessions_with_features	3
1	sessions_with_tensors	1
2	sessions_with_embeddings	1
3	sessions_ready_for_full_neural_analysis	1
4	minimum_target_for_top_journal_rerun	3

Missing asset to-do list

Use this table to process additional sessions before rerunning notebooks 13–17.

```
todo_rows = []

for _, row in asset_index.iterrows():
    session_id = str(row["session_id"])
```

```
    if not row.get("has_tensor", False):
        todo_rows.append({
            "session_id": session_id,
            "missing_asset": "tensor",
            "recommended_action": "Run notebooks 02–03 or scripts/preprocess
            "expected_output": f"data/interim/session_{session_id}_natural_m
        })
    if not row.get("has_embeddings", False):
        todo_rows.append({
            "session_id": session_id,
            "missing_asset": "embeddings",
            "recommended_action": "Run notebooks 05–07 or scripts/run_manifo
            "expected_output": f"data/processed/session_{session_id}_embeddi
        })
    if not row.get("has_features", False):
        todo_rows.append({
            "session_id": session_id,
            "missing_asset": "enhanced_features",
            "recommended_action": "Run notebook 12 with WRITE_PUBLICATION_OU
            "expected_output": f"data/processed/session_{session_id}_publica
        })

todo = pd.DataFrame(todo_rows)
save_table(todo, paths.publication_tables_dir / "12c_missing_multisession_as
display(todo)

if int(asset_index["ready_for_full_neural_analysis"].sum()) >= MIN_TOP_JOURN
    print("Top-journal multi-session asset threshold is met. Rerun notebooks
else:
    print("Top-journal multi-session asset threshold is not met yet.")
    print("Create tensors and embeddings for more sessions, then rerun noteb
```

	session_id	missing_asset	recommended_action	expected_c
0	500964514	tensor	Run notebooks 02–03 or scripts/ preprocess_sess...	data/i session_500964514_natural_mov
1	500964514	embeddings	Run notebooks 05–07 or scripts/ run_manifold_an...	data/proc session_500964514_embeddin
2	501271265	tensor	Run notebooks 02–03 or scripts/	data/i

Optional command launcher for existing project scripts

This is intentionally off by default because project script command-line arguments may differ. Set `RUN_EXISTING_PROJECT_SCRIPTS=1` only after confirming your

scripts support the proposed commands.

```
import subprocess

RUN_EXISTING_PROJECT_SCRIPTS = os.environ.get("RUN_EXISTING_PROJECT_SCRIPTS"

commands = []
for _, row in todo.iterrows() if not todo.empty else []:
    session_id = str(row["session_id"])
    if row["missing_asset"] == "tensor":
        commands.append([sys.executable, str(PROJECT_ROOT / "scripts" / "pre
    elif row["missing_asset"] == "embeddings":
        commands.append([sys.executable, str(PROJECT_ROOT / "scripts" / "run

command_table = pd.DataFrame({"command": [" ".join(cmd) for cmd in commands]
save_table(command_table, paths.publication_tables_dir / "12c_proposed_asset
display(command_table)

if RUN_EXISTING_PROJECT_SCRIPTS:
    for cmd in commands:
        print("Running:", " ".join(cmd))
        subprocess.run(cmd, cwd=PROJECT_ROOT, check=False)
else:
    print("Dry run only. Set RUN_EXISTING_PROJECT_SCRIPTS=1 to run proposed
```

	command
0	c:\Users\Peter\.neuro\Scripts\python.exe C:\Us...
1	c:\Users\Peter\.neuro\Scripts\python.exe C:\Us...
2	c:\Users\Peter\.neuro\Scripts\python.exe C:\Us...
3	c:\Users\Peter\.neuro\Scripts\python.exe C:\Us...

```
Dry run only. Set RUN_EXISTING_PROJECT_SCRIPTS=1 to run proposed commands.
```

```
# ----------------------------------------------------------------------
# Check script help with correct PYTHONPATH
# ----------------------------------------------------------------------

import subprocess
import sys
import os
from pathlib import Path

PROJECT_ROOT = Path(
    r"C:\Users\Peter\Documents\projects\NeuroAI\latent-manifold-v1-natural-m
).resolve()
```

```
SRC_DIR = PROJECT_ROOT / "src"

env = os.environ.copy()
env["PYTHONPATH"] = str(SRC_DIR) + os.pathsep + env.get("PYTHONPATH", "")

for script_name in ["preprocess_sessions.py", "run_manifold_analysis.py"]:
    script_path = PROJECT_ROOT / "scripts" / script_name

    print("\n" + "=" * 90)
    print(script_path)
    print("=" * 90)

    result = subprocess.run(
        [sys.executable, str(script_path), "--help"],
        cwd=PROJECT_ROOT,
        env=env,
        capture_output=True,
        text=True,
    )

    print("Return code:", result.returncode)

    if result.stdout:
        print("\nSTDOUT:")
        print(result.stdout)

    if result.stderr:
        print("\nSTDERR:")
        print(result.stderr)
```

```
==========================================================================================
C:\Users\Peter\Documents\projects\NeuroAI\latent-manifold-v1-natural-movies\s
==========================================================================================
Return code: 0

STDOUT:
usage: preprocess_sessions.py [-h] [--config CONFIG]

options:
  -h, --help       show this help message and exit
  --config CONFIG


==========================================================================================
C:\Users\Peter\Documents\projects\NeuroAI\latent-manifold-v1-natural-movies\s
==========================================================================================
Return code: 0

STDOUT:
usage: run_manifold_analysis.py [-h] [--config CONFIG]
```

```
options:
  -h, --help       show this help message and exit
  --config CONFIG
```

```
command_path = paths.publication_tables_dir / "12c_proposed_asset_generation

commands = pd.read_csv(command_path)
pd.set_option("display.max_colwidth", None)
display(commands)
```

	command
0	c:\Users\Peter\.neuro\Scripts\python.exe C:\Users\Peter\Documents\projects\NeuroAI\latent-manifold-v1-natural-movies\scripts\preprocess_sessions.py --session-id 500964514
1	c:\Users\Peter\.neuro\Scripts\python.exe C:\Users\Peter\Documents\projects\NeuroAI\latent-manifold-v1-natural-movies\scripts\run_manifold_analysis.py --session-id 500964514
2	c:\Users\Peter\.neuro\Scripts\python.exe C:\Users\Peter\Documents\projects\NeuroAI\latent-manifold-v1-natural-movies\scripts\preprocess_sessions.py --session-id 501271265
3	c:\Users\Peter\.neuro\Scripts\python.exe C:\Users\Peter\Documents\projects\NeuroAI\latent-manifold-v1-natural-movies\scripts\run_manifold_analysis.py --session-id 501271265

```
# ------------------------------------------------------------------------
# Run proposed tensor/embedding generation commands with fixed PYTHONPATH
# ------------------------------------------------------------------------

import subprocess
import sys
import os
import shlex
import pandas as pd
from pathlib import Path

PROJECT_ROOT = Path(
    r"C:\Users\Peter\Documents\projects\NeuroAI\latent-manifold-v1-natural-m
```

```
).resolve()

SRC_DIR = PROJECT_ROOT / "src"

env = os.environ.copy()
env["PYTHONPATH"] = str(SRC_DIR) + os.pathsep + env.get("PYTHONPATH", "")

command_path = paths.publication_tables_dir / "12c_proposed_asset_generation

commands_df = pd.read_csv(command_path)

run_logs = []

for i, row in commands_df.iterrows():
    command_text = row["command"]

    print("\n" + "=" * 90)
    print(f"Running command {i + 1}/{len(commands_df)}")
    print(command_text)
    print("=" * 90)

    # Windows-safe parsing.
    cmd = shlex.split(command_text, posix=False)

    result = subprocess.run(
        cmd,
        cwd=PROJECT_ROOT,
        env=env,
        capture_output=True,
        text=True,
    )

    print("Return code:", result.returncode)

    if result.stdout:
        print("\nSTDOUT:")
        print(result.stdout)

    if result.stderr:
        print("\nSTDERR:")
        print(result.stderr)

    run_logs.append({
        "command_index": i,
        "command": command_text,
        "return_code": result.returncode,
        "stdout_tail": result.stdout[-4000:] if result.stdout else "",
        "stderr_tail": result.stderr[-4000:] if result.stderr else "",
    })

run_logs = pd.DataFrame(run_logs)
```

```
run_logs = pd.DataFrame(run_logs)

save_table(
    run_logs,
    paths.publication_tables_dir / "12c_asset_generation_command_run_logs.cs
)

display(run_logs)
```

```
================================================================================
Running command 1/4
c:\Users\Peter\.neuro\Scripts\python.exe C:\Users\Peter\Documents\projects\Ne
================================================================================
Return code: 2

STDERR:
usage: preprocess_sessions.py [-h] [--config CONFIG]
preprocess_sessions.py: error: unrecognized arguments: --session-id 500964514


================================================================================
Running command 2/4
c:\Users\Peter\.neuro\Scripts\python.exe C:\Users\Peter\Documents\projects\Ne
================================================================================
Return code: 2

STDERR:
usage: run_manifold_analysis.py [-h] [--config CONFIG]
run_manifold_analysis.py: error: unrecognized arguments: --session-id 5009645


================================================================================
Running command 3/4
c:\Users\Peter\.neuro\Scripts\python.exe C:\Users\Peter\Documents\projects\Ne
================================================================================
Return code: 2

STDERR:
usage: preprocess_sessions.py [-h] [--config CONFIG]
preprocess_sessions.py: error: unrecognized arguments: --session-id 501271265


================================================================================
Running command 4/4
c:\Users\Peter\.neuro\Scripts\python.exe C:\Users\Peter\Documents\projects\Ne
================================================================================
Return code: 2

STDERR:
usage: run_manifold_analysis.py [-h] [--config CONFIG]
run_manifold_analysis.py: error: unrecognized arguments: --session-id 5012712
```

command index	command	return code	stdou

0	0	c:\Users\Peter\.neuro\Scripts\python.exe C:\Users\Peter\Documents\projects\NeuroAI\latent-manifold-v1-natural-movies\scripts\preprocess_sessions.py --session-id 500964514	2
1	1	c:\Users\Peter\.neuro\Scripts\python.exe C:\Users\Peter\Documents\projects\NeuroAI\latent-manifold-v1-natural-movies\scripts\run_manifold_analysis.py --session-id 500964514	2

```
# ----------------------------------------------------------------------
# Create per-session publication configs and run tensor/embedding generation
# ----------------------------------------------------------------------
# Your scripts only accept --config, not --session-id.
# This cell creates temporary config files for missing sessions and runs:
#   scripts/preprocess_sessions.py --config <session_config>
#   scripts/run_manifold_analysis.py --config <session_config>
# ----------------------------------------------------------------------

PROJECT_ROOT = Path(
    r"C:\Users\Peter\Documents\projects\NeuroAI\latent-manifold-v1-natural-m
).resolve()

SRC_DIR = PROJECT_ROOT / "src"
CONFIG_DIR = PROJECT_ROOT / "configs"
SCRIPT_DIR = PROJECT_ROOT / "scripts"

BASE_CONFIG = CONFIG_DIR / "default.yaml"

if not PROJECT_ROOT.exists():
    raise FileNotFoundError(f"Project root does not exist: {PROJECT_ROOT}")

if not BASE_CONFIG.exists():
    raise FileNotFoundError(f"Base config not found: {BASE_CONFIG}")

env = os.environ.copy()
env["PYTHONPATH"] = str(SRC_DIR) + os.pathsep + env.get("PYTHONPATH", "")

# Missing sessions from your readiness output.
missing_sessions = ["500964514", "501271265"]

print("Missing sessions to process:", missing_sessions)
print("Base config:", BASE_CONFIG)


def recursive_set_session_ids(obj, session_id):
    """
    Recursively modify common session-selection keys in a config dictionary.
```

```
    Recursively modify common session selection keys in a config dictionary.
    This is intentionally broad because project configs often use different
    names such as session_ids, experiment_ids, selected_experiment_ids, etc.
    """
    if isinstance(obj, dict):
        out = {}

        for key, value in obj.items():
            key_lower = str(key).lower()

            if key_lower in {
                "session_id",
                "ophys_session_id",
                "experiment_id",
                "ophys_experiment_id",
                "selected_session_id",
                "selected_experiment_id",
            }:
                out[key] = int(session_id)

            elif key_lower in {
                "session_ids",
                "ophys_session_ids",
                "experiment_ids",
                "ophys_experiment_ids",
                "selected_session_ids",
                "selected_experiment_ids",
                "include_sessions",
                "include_experiments",
            }:
                out[key] = [int(session_id)]

            else:
                out[key] = recursive_set_session_ids(value, session_id)

        return out

    if isinstance(obj, list):
        return [recursive_set_session_ids(x, session_id) for x in obj]

    return obj


def add_publication_session_overrides(cfg, session_id):
    """
    Adds explicit top-level override fields. If your scripts know any of the
    keys, they will use them. If not, they are harmless metadata.
    """
    cfg = dict(cfg)

    cfg["publication_session_override"] = {
```

```
            "session_id": int(session_id),
            "ophys_experiment_id": int(session_id),
            "reason": "Generated by notebook 12c for multi-session publication u
        }

        # Common project sections.
        cfg.setdefault("data", {})
        cfg["data"]["session_id"] = int(session_id)
        cfg["data"]["session_ids"] = [int(session_id)]
        cfg["data"]["experiment_id"] = int(session_id)
        cfg["data"]["experiment_ids"] = [int(session_id)]
        cfg["data"]["ophys_experiment_id"] = int(session_id)
        cfg["data"]["ophys_experiment_ids"] = [int(session_id)]

        cfg.setdefault("allen", {})
        cfg["allen"]["session_id"] = int(session_id)
        cfg["allen"]["session_ids"] = [int(session_id)]
        cfg["allen"]["experiment_id"] = int(session_id)
        cfg["allen"]["experiment_ids"] = [int(session_id)]
        cfg["allen"]["ophys_experiment_id"] = int(session_id)
        cfg["allen"]["ophys_experiment_ids"] = [int(session_id)]

        cfg.setdefault("cohort", {})
        cfg["cohort"]["session_ids"] = [int(session_id)]
        cfg["cohort"]["experiment_ids"] = [int(session_id)]
        cfg["cohort"]["ophys_experiment_ids"] = [int(session_id)]
        cfg["cohort"]["max_sessions"] = 1

        return cfg


    # ---------------------------------------------------------------------
    # Create temporary configs
    # ---------------------------------------------------------------------

    with BASE_CONFIG.open("r", encoding="utf-8") as f:
        base_cfg = yaml.safe_load(f)

    session_config_paths = []

    for session_id in missing_sessions:
        cfg_session = recursive_set_session_ids(base_cfg, session_id)
        cfg_session = add_publication_session_overrides(cfg_session, session_id)

        out_config = CONFIG_DIR / f"publication_session_{session_id}.yaml"

        with out_config.open("w", encoding="utf-8") as f:
            yaml.safe_dump(cfg_session, f, sort_keys=False)

        session_config_paths.append(out_config)
```

```
    print("Wrote:", out_config)


# ---------------------------------------------------------------------
# Run scripts
# ---------------------------------------------------------------------

commands = []

for session_id, cfg_path in zip(missing_sessions, session_config_paths):
    commands.append({
        "session_id": session_id,
        "stage": "preprocess",
        "command": [
            sys.executable,
            str(SCRIPT_DIR / "preprocess_sessions.py"),
            "--config",
            str(cfg_path),
        ],
    })

    commands.append({
        "session_id": session_id,
        "stage": "manifold",
        "command": [
            sys.executable,
            str(SCRIPT_DIR / "run_manifold_analysis.py"),
            "--config",
            str(cfg_path),
        ],
    })

run_logs = []

for item in commands:
    session_id = item["session_id"]
    stage = item["stage"]
    cmd = item["command"]

    print("\n" + "=" * 100)
    print(f"Session {session_id} | Stage: {stage}")
    print(" ".join(cmd))
    print("=" * 100)

    result = subprocess.run(
        cmd,
        cwd=PROJECT_ROOT,
        env=env,
        capture_output=True,
        text=True,
```

```
        )

        print("Return code:", result.returncode)

        if result.stdout:
            print("\nSTDOUT:")
            print(result.stdout[-5000:])

        if result.stderr:
            print("\nSTDERR:")
            print(result.stderr[-5000:])

        run_logs.append({
            "session_id": session_id,
            "stage": stage,
            "return_code": result.returncode,
            "command": " ".join(cmd),
            "stdout_tail": result.stdout[-5000:] if result.stdout else "",
            "stderr_tail": result.stderr[-5000:] if result.stderr else "",
        })

    run_logs = pd.DataFrame(run_logs)

    save_table(
        run_logs,
        paths.publication_tables_dir / "12c_per_session_asset_generation_logs.cs
    )

    display(run_logs)


    # ------------------------------------------------------------------
    # Verify expected outputs
    # ------------------------------------------------------------------

    verification_rows = []

    for session_id in missing_sessions:
        expected_tensor = PROJECT_ROOT / "data" / "interim" / f"session_{session
        expected_embedding = PROJECT_ROOT / "data" / "processed" / f"session_{se

        verification_rows.append({
            "session_id": session_id,
            "expected_tensor": str(expected_tensor),
            "tensor_exists": expected_tensor.exists(),
            "expected_embedding": str(expected_embedding),
            "embedding_exists": expected_embedding.exists(),
        })

    verification = pd.DataFrame(verification_rows)
```

```
save_table(
    verification,
    paths.publication_tables_dir / "12c_per_session_asset_generation_verific
)

display(verification)
```

```
Missing sessions to process: ['500964514', '501271265']
Base config: C:\Users\Peter\Documents\projects\NeuroAI\latent-manifold-v1-nat
Wrote: C:\Users\Peter\Documents\projects\NeuroAI\latent-manifold-v1-natural-m
Wrote: C:\Users\Peter\Documents\projects\NeuroAI\latent-manifold-v1-natural-m

================================================================================
Session 500964514 | Stage: preprocess
c:\Users\Peter\.neuro\Scripts\python.exe C:\Users\Peter\Documents\projects\Ne
================================================================================
Return code: 0

STDOUT:
Preprocessing experiment 500855614
Retained 163 / 164 cells
Saved C:\Users\Peter\Documents\projects\NeuroAI\latent-manifold-v1-natural-mo
Preprocessing experiment 500964514
Retained 214 / 214 cells
Saved C:\Users\Peter\Documents\projects\NeuroAI\latent-manifold-v1-natural-mo
Preprocessing experiment 501271265
Retained 215 / 215 cells
Saved C:\Users\Peter\Documents\projects\NeuroAI\latent-manifold-v1-natural-mo


STDERR:
c:\Users\Peter\.neuro\Lib\site-packages\allensdk\core\brain_observatory_nwb_d
  from pkg_resources import parse_version


================================================================================
Session 500964514 | Stage: manifold
c:\Users\Peter\.neuro\Scripts\python.exe C:\Users\Peter\Documents\projects\Ne
================================================================================
Exception in thread Thread-22 (_readerthread):
Traceback (most recent call last):
  File "C:\Users\Peter\AppData\Local\Python\pythoncore-3.11-64\Lib\threading.
    self.run()
  File "C:\Users\Peter\AppData\Local\Python\pythoncore-3.11-64\Lib\threading.
    self._target(*self._args, **self._kwargs)
  File "C:\Users\Peter\AppData\Local\Python\pythoncore-3.11-64\Lib\subprocess
    buffer.append(fh.read())
                  ^^^^^^^^^
  File "C:\Users\Peter\AppData\Local\Python\pythoncore-3.11-64\Lib\encodings\
    return codecs.charmap_decode(input,self.errors,decoding_table)[0]
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
UnicodeDecodeError: 'charmap' codec can't decode byte 0x8f in position 4888:
Return code: 1
```

```
================================================================================
Session 501271265 | Stage: preprocess
c:\Users\Peter\.neuro\Scripts\python.exe C:\Users\Peter\Documents\projects\Ne
================================================================================
Return code: 0

STDOUT:
Preprocessing experiment 500855614
Retained 163 / 164 cells
Saved C:\Users\Peter\Documents\projects\NeuroAI\latent-manifold-v1-natural-mo
Preprocessing experiment 500964514
Retained 214 / 214 cells
Saved C:\Users\Peter\Documents\projects\NeuroAI\latent-manifold-v1-natural-mo
Preprocessing experiment 501271265
Retained 215 / 215 cells
Saved C:\Users\Peter\Documents\projects\NeuroAI\latent-manifold-v1-natural-mo


STDERR:
c:\Users\Peter\.neuro\Lib\site-packages\allensdk\core\brain_observatory_nwb_d
  from pkg_resources import parse_version


================================================================================
Session 501271265 | Stage: manifold
c:\Users\Peter\.neuro\Scripts\python.exe C:\Users\Peter\Documents\projects\Ne
================================================================================
Exception in thread Thread-26 (_readerthread):
Traceback (most recent call last):
  File "C:\Users\Peter\AppData\Local\Python\pythoncore-3.11-64\Lib\threading.
    self.run()
  File "C:\Users\Peter\AppData\Local\Python\pythoncore-3.11-64\Lib\threading.
    self._target(*self._args, **self._kwargs)
  File "C:\Users\Peter\AppData\Local\Python\pythoncore-3.11-64\Lib\subprocess
    buffer.append(fh.read())
                  ^^^^^^^^^
  File "C:\Users\Peter\AppData\Local\Python\pythoncore-3.11-64\Lib\encodings\
    return codecs.charmap_decode(input,self.errors,decoding_table)[0]
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
UnicodeDecodeError: 'charmap' codec can't decode byte 0x8f in position 5001:
Return code: 1
```

	session_id	stage	return_code	co
0	500964514	preprocess	0	c:\Users\Peter\.neuro\Scripts\python. \Users\Peter\Documents\projects\NeuroAI\ manifold-v1-n movies\scripts\preprocess_sessions.py --co \Users\Peter\Documents\projects\NeuroAI\ manifold-v1-n movies\configs\publication_session_50096451

c:\Users\Peter\.neuro\Scripts\python.

1	500964514	manifold	1	\Users\Peter\Documents\projects\NeuroAI\manifold-v1-n movies\scripts\run_manifold_analysis.py --co \Users\Peter\Documents\projects\NeuroAI\manifold-v1-n movies\configs\publication_session_50096451
2	501271265	preprocess	0	c:\Users\Peter\.neuro\Scripts\python. \Users\Peter\Documents\projects\NeuroAI\manifold-v1-n movies\scripts\preprocess_sessions.py --co \Users\Peter\Documents\projects\NeuroAI\manifold-v1-n movies\configs\publication_session_50127126
3	501271265	manifold	1	c:\Users\Peter\.neuro\Scripts\python. \Users\Peter\Documents\projects\NeuroAI\manifold-v1-n movies\scripts\run_manifold_analysis.py --co \Users\Peter\Documents\projects\NeuroAI\manifold-v1-n movies\configs\publication_session_50127126

	session_id	expected_tensor	te
0	500964514	C:\Users\Peter\Documents\projects\NeuroAI\latent-manifold-v1-natural-movies\data\interim\session_500964514_natural_movie_one_tensor.h5	
1	501271265	C:\Users\Peter\Documents\projects\NeuroAI\latent-manifold-v1-natural-movies\data\interim\session_501271265_natural_movie_one_tensor.h5	